Guía 3 de 4 — Secretos (.env) y datos personales

DPL1046 · Clase 5 · Para repasar en casa, a tu ritmo.

Cuando trabajas con claves de servicios y con datos de personas, tienes que cuidarlos desde el primer proyecto. Esta guía repasa dos ideas del bloque de seguridad: **guardar secretos fuera del código** (`.env`) y **proteger datos personales** (hashing y masking).

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado. Si tu computador usa `requests` por primera vez, instala lo de la Clase 4: `pip install -r clases/clase-04/material/requirements.txt` .

---

1. Secretos: la clave no va en el código

Algunas APIs piden una **clave** (API key). Esa clave es como la **llave de tu casa**: si la dejas escrita en el código y subes el código a GitHub, cualquiera puede usarla. Regla de oro: **nunca escribas una clave en un archivo `.py`** .

La solución: un archivo aparte llamado **`.env`** , que vive en tu computador y **Git ignora**. Piensa en una **caja fuerte** al lado de la casa: el script sabe abrirla, pero la clave nunca queda a la vista.

Paso 1: crear el archivo `.env` (en la carpeta raíz del repo)

```
API_KEY=mi_clave_de_ejemplo_1234
```

Una variable por línea, `NOMBRE=valor` , **sin comillas ni espacios** alrededor del `=` .

Paso 2: leerlo desde Python

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()                          # abre la caja fuerte: lee el .env
clave = os.getenv("API_KEY")           # saca el valor de API_KEY

if clave is None:
    print("No encontré API_KEY. Crea el archivo .env.")
else:
    print("Clave cargada. Termina en:", clave[-4:])
    cabeceras = {"Authorization": "Bearer " + clave}
    r = requests.get("https://mindicador.cl/api/dolar", headers=cabeceras,
timeout=10)
    print(r.json()["serie"][0])
```

- `load_dotenv()` lee el `.env` y deja disponibles sus variables.
- `os.getenv("API_KEY")` devuelve el valor, o `None` (“nada”) si la variable no existe.
- `if clave is None:` es una **decisión**: lo que va con sangría debajo se ejecuta solo si no hay clave.
- `clave[-4:]` toma los **últimos cuatro caracteres**: confirmamos que cargó **sin mostrar la clave completa** (como las boletas que muestran `**** 1234`).
- `{"Authorization": "Bearer " + clave}` es el formato estándar para mandar una clave; `headers=cabeceras` la envía junto con el pedido, como mostrar tu credencial en la entrada de un edificio.

> **Ojo:** mindicador no pide clave. Practicamos el patrón exacto que usarías con una API que sí la pida.

Checklist de seguridad

- ☐ La clave está en `.env`, no en un `.py`.
- ☐ `.env` aparece en el `.gitignore`.
- ☐ Nunca imprimo la clave completa.
- ☐ Si una clave se filtró por error, la **revoco** y creo una nueva.

---

2. Datos personales (PII)

PII es información que identifica a una persona: RUT, nombre, correo, teléfono, dirección. En Chile la protege la **Ley 19.628**. Si puedes evitarlo, **no trabajes con RUT y correos “en crudo”**; y antes de enviarlos a un sistema externo o a una IA, protégelos.

Dos técnicas, con ejemplos de la vida diaria:

- **Hashing = una licuadora.** Si mezclas fruta en una licuadora, no puedes volver a separarla; pero la **misma fruta** da siempre el **mismo jugo**. Con un RUT: se convierte en un código que **no se puede revertir**, y el mismo RUT da siempre el mismo código. Así puedes **cruzar tablas** sin guardar el RUT real.
- **Masking = tapar con el dedo.** Como la boleta que muestra solo `**** 1234`: se ve una **parte** del dato.

```
import hashlib

SALT = "udla-dpl1046"
rut = "12.345.678-9"
email = "juan@gmail.com"

# Hashing
texto = (SALT + rut).encode("utf-8")
rut_hash = hashlib.sha256(texto).hexdigest()[:16]

# Masking
usuario, dominio = email.split("@")
email_masked = usuario[0] + "***@" + dominio

print(rut_hash)        # un código de 16 caracteres
print(email_masked)    # j***@gmail.com
```

- `SALT` es la **sal**: un texto secreto que se mezcla con el dato antes del hash, para que sea mucho más difícil adivinar el original. En un proyecto real va en el `.env`.
- `.encode("utf-8")` convierte el texto a **bytes**, que es lo que recibe la licuadora.
- `sha256(...).hexdigest()` calcula el código irreversible; `[:16]` se queda con los 16 primeros caracteres.
- `email.split("@")` **parte** el texto en la arroba: `juan` y `gmail.com`, guardados en dos variables a la vez.
- `usuario[0] + "***@" + dominio` une la primera letra, tres asteriscos y el dominio.

Técnica	¿Se puede revertir?	¿Sirve para unir tablas?	Ejemplo
Hashing	No	Sí	RUT → `a3f9...`
Masking	No (se pierde parte)	No	`juan@...` → `j***@...`

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`clave is None`	No existe `.env`, está en otra carpeta, o el nombre de la variable no coincide	El `.env` va en la raíz del repo; revisa `API_KEY`
`ModuleNotFoundError: dotenv`	Falta instalar	`pip install python-dotenv` con el `.venv` activo
El `.env` aparece en `git status`	No está en `.gitignore`	Agrégalo **antes** de hacer commit
`KeyError` / `ConnectionError` al usar la clave	Sin internet o respuesta distinta a la esperada	Revisa tu conexión; imprime el JSON recibido
El hash cambia entre ejecuciones	Cambió el `SALT` o el texto original	Usa siempre el mismo `SALT` y el mismo formato de RUT

Practica tú

1. ¿Qué devuelve `os.getenv("NO_EXISTE")`?
2. ¿Por qué el `.env` no se sube a Git?
3. Enmascara el correo `maria@correo.cl` con el mismo método.
4. ¿Qué técnica usarías para poder **unir dos tablas por RUT** sin guardar el RUT real?

▼ Ver respuestas

1. `None`, porque la variable no existe.
2. Porque contiene secretos; está en el `.gitignore` para que Git lo ignore.
3. `usuario, dominio = "maria@correo.cl".split("@")` → `usuario[0] + "***@" + dominio` → `m***@correo.cl`.
4. **Hashing** (con un `SALT`): el mismo RUT da siempre el mismo código, y no se puede revertir.

Para llevarte

- **Las claves van en `.env`**, nunca en el código, y nunca se imprimen completas.
- `load_dotenv()` abre la caja fuerte; `os.getenv("NOMBRE")` saca el valor (o `None`).
- **Hashing** para poder cruzar tablas sin guardar el dato real; **masking** para mostrar solo una parte.
- En tu proyecto cae en la slide 7 (**Buenas prácticas**): “sin credenciales en el código” vale el 15% de la rúbrica.